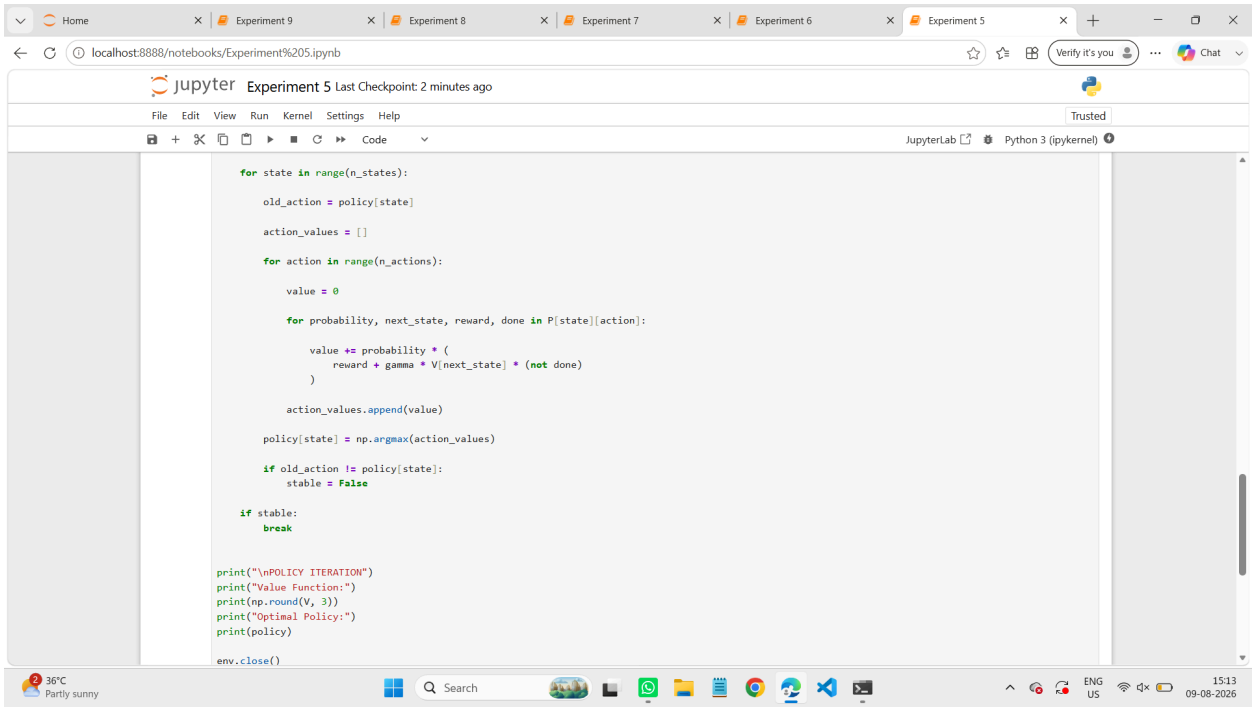
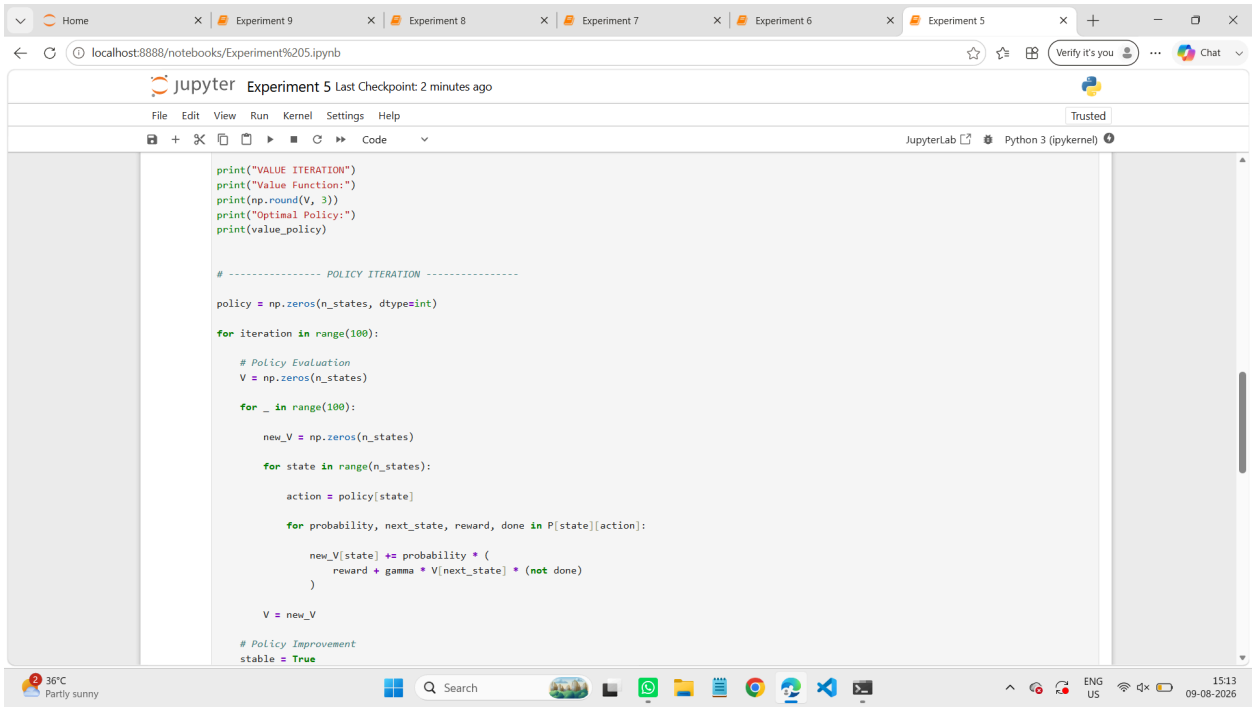
Experiment 5 RL

Name: G.Vinay Kumar Reddy **Reg No:**192472093

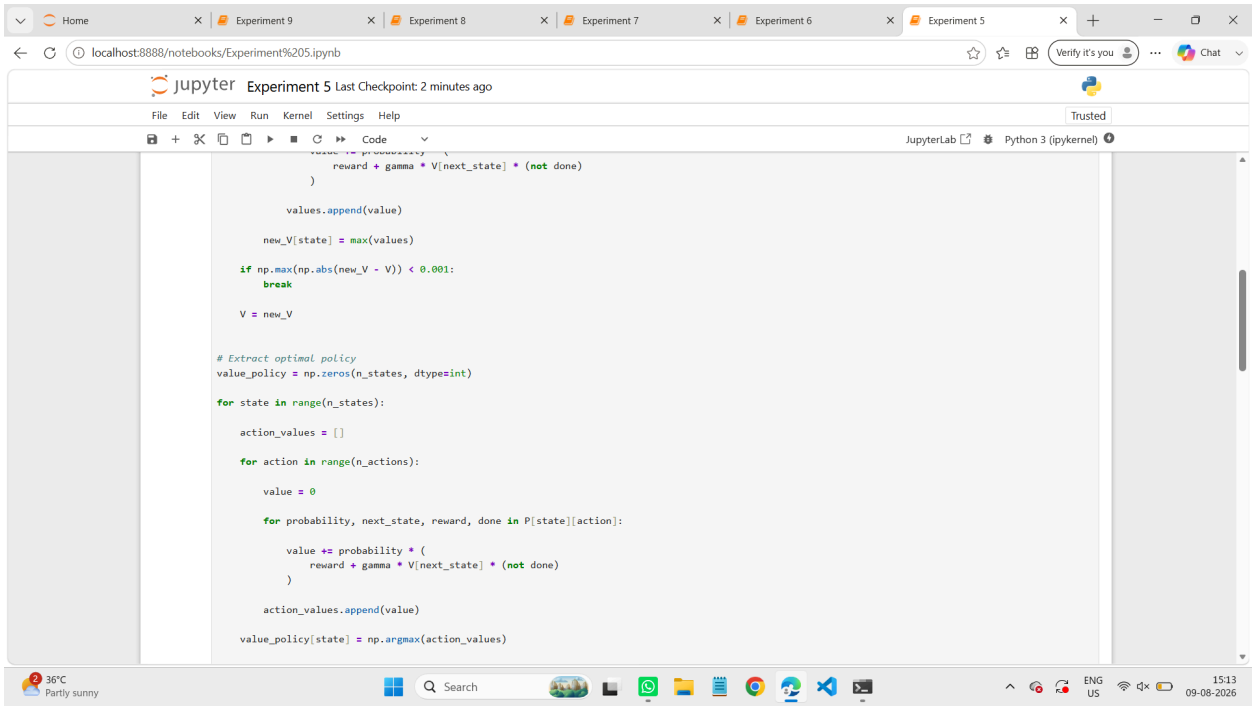
Course Code: MLA0305 **Slot:** B

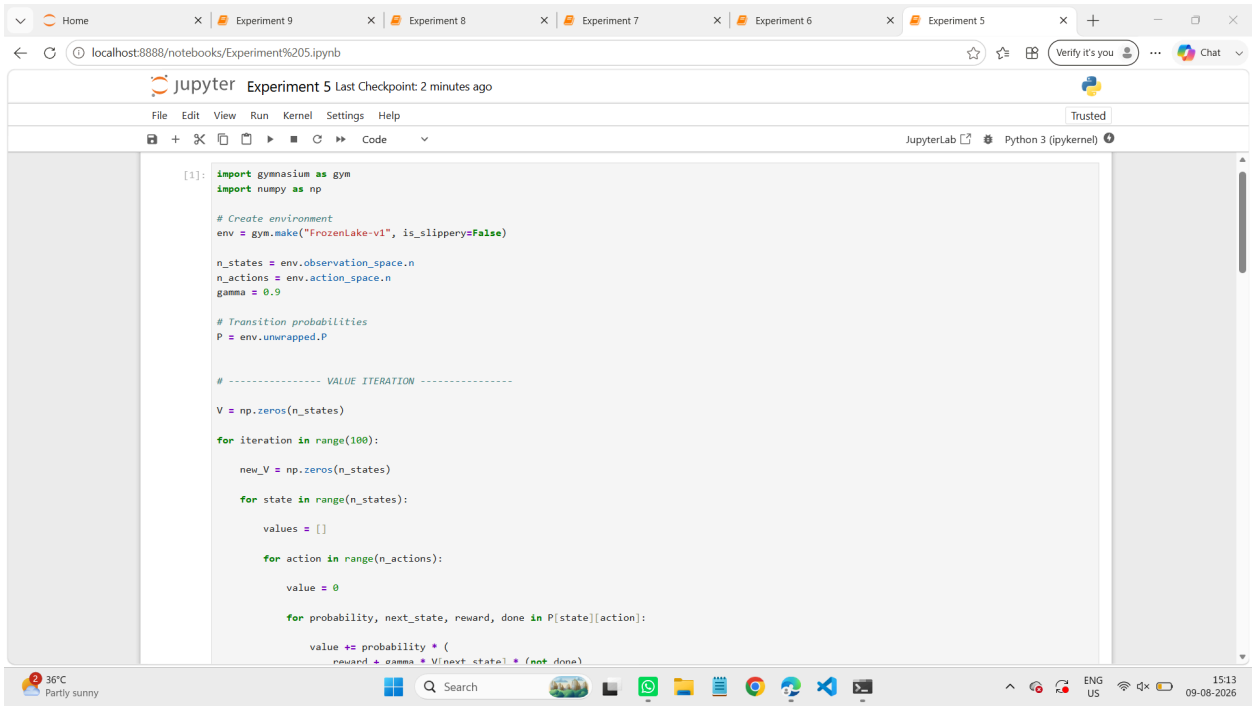


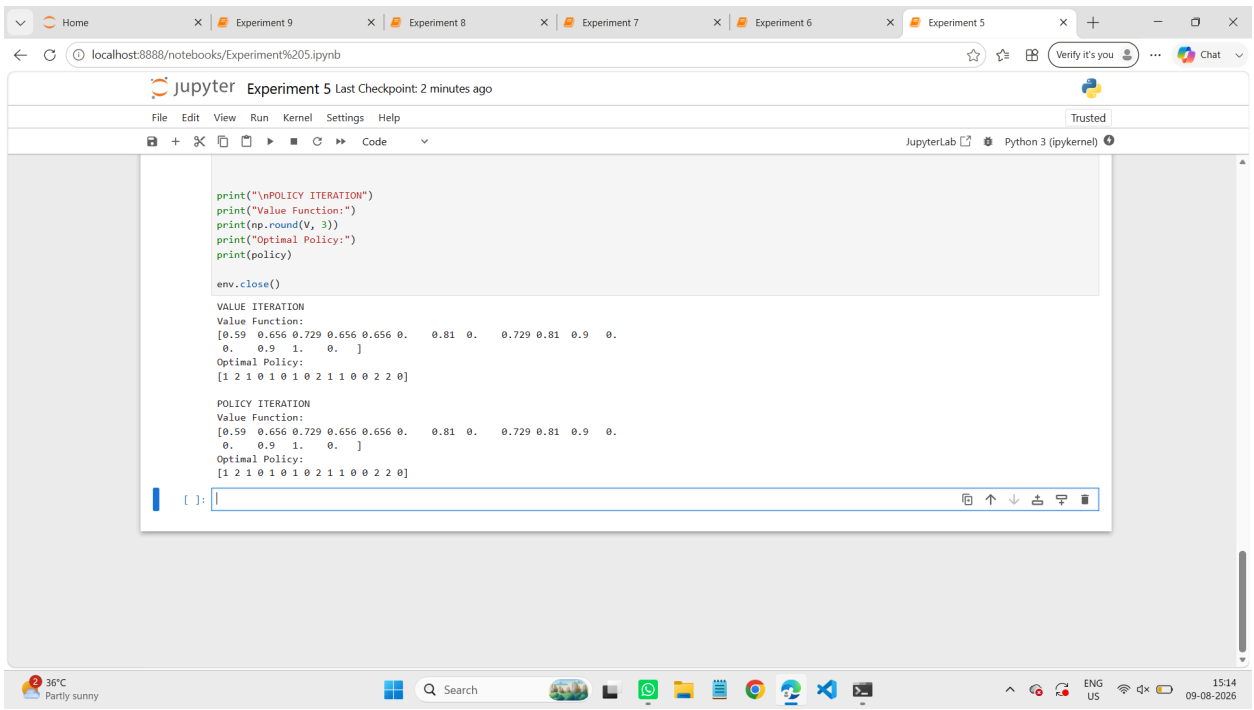
```
    for state in range(n_states):

        old_action = policy[state]

        action_values = []

        for action in range(n_actions):

            value = 0

            for probability, next_state, reward, done in P[state][action]:

                value += probability * (
                    reward + gamma * V[next_state] * (not done)
                )

            action_values.append(value)

        policy[state] = np.argmax(action_values)

        if old_action != policy[state]:
            stable = False

    if stable:
        break


print("\nPOLICY ITERATION")
print("Value Function:")
print(np.round(V, 3))
print("Optimal Policy:")
print(policy)

env.close()
```



```
print("VALUE ITERATION")
print("Value Function:")
print(np.round(V, 3))
print("Optimal Policy:")
print(value_policy)


# --------------- POLICY ITERATION ---------------

policy = np.zeros(n_states, dtype=int)

for iteration in range(100):

    # Policy Evaluation
    V = np.zeros(n_states)

    for _ in range(100):

        new_V = np.zeros(n_states)

        for state in range(n_states):

            action = policy[state]

            for probability, next_state, reward, done in P[state][action]:

                new_V[state] += probability * (
                    reward + gamma * V[next_state] * (not done)
                )

        V = new_V

    # Policy Improvement
    stable = True
```


Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
Experiment 5
localhost:8888/notebooks/Experiment%205.ipynb
Verify it's you
Chat
jupyter
Experiment 5 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
reward + gamma * V[next_state] * (not done)
)
values.append(value)
new_V[state] = max(values)
if np.max(np.abs(new_V - V)) < 0.001:
break
V = new_V
# Extract optimal policy
value_policy = np.zeros(n_states, dtype=int)
for state in range(n_states):
action_values = []
for action in range(n_actions):
value = 0
for probability, next_state, reward, done in P[state][action]:
value += probability * (
reward + gamma * V[next_state] * (not done)
)
action_values.append(value)
value_policy[state] = np.argmax(action_values)
36°C
Partly sunny
Search
ENG
US
15:13
09-08-2026


Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
Experiment 5
localhost:8888/notebooks/Experiment%205.ipynb
jupyter Experiment 5 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]:
import gymnasium as gym
import numpy as np
# Create environment
env = gym.make("FrozenLake-v1", is_slippery=False)
n_states = env.observation_space.n
n_actions = env.action_space.n
gamma = 0.9
# Transition probabilities
P = env.unwrapped.P
# --------------- VALUE ITERATION ---------------
V = np.zeros(n_states)
for iteration in range(100):
new_V = np.zeros(n_states)
for state in range(n_states):
values = []
for action in range(n_actions):
value = 0
for probability, next_state, reward, done in P[state][action]:
value += probability * (
36°C
Partly sunny
Search
ENG
US
15:13
09-08-2026


Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
Experiment 5
localhost:8888/notebooks/Experiment%205.ipynb
jupyter Experiment 5 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
print("\nPOLICY ITERATION")
print("Value Function:")
print(np.round(V, 3))
print("Optimal Policy:")
print(policy)
env.close()
VALUE ITERATION
Value Function:
[0.59 0.656 0.729 0.656 0.656 0. 0.81 0. 0.729 0.81 0.9 0.
0. 0.9 1. 0.]
Optimal Policy:
[1 2 1 0 1 0 1 0 2 1 1 0 0 2 2 0]
POLICY ITERATION
Value Function:
[0.59 0.656 0.729 0.656 0.656 0. 0.81 0. 0.729 0.81 0.9 0.
0. 0.9 1. 0.]
Optimal Policy:
[1 2 1 0 1 0 1 0 2 1 1 0 0 2 2 0]
[ ]:
36°C
Partly sunny
Search
ENG
US
15:14
09-08-2026